

Document Similarity Analyzer Using Suffix Structures

Billakanti Siri	2520030601
Bhavana Kondakrindi	2520030082
Aniketh Pani	2520030184

**Under the Esteemed Guidance of
Dr Anuradha (CSE)**

Table Of Contents

01

Introduction

02

Abstract

03

Literature Survey

04

Problem Statement and objectives

05

Hardware and Software Requirements

05

Results

06

Conclusion

07

Future Scope

Abstract

DOCS_SIMILARITY is a document similarity analyzer designed to compare multiple large text documents and identify duplicate or highly similar documents.

The system uses Suffix Arrays and LCP (Longest Common Prefix) Arrays to analyze common substrings and matching patterns between documents. The Kasai Algorithm is used for efficient LCP construction.

The identified common textual content is used to calculate similarity scores and rank document pairs based on their similarity.

Applications:

Plagiarism Detection • Duplicate Document Detection • Document Clustering

Introduction

- Comparing large text documents manually is time-consuming and inefficient.
- Traditional approaches may require repeated comparisons between documents.
- DOCS_SIMILARITY uses advanced string-processing techniques to efficiently identify shared textual patterns.
- Suffix Arrays organize suffixes of the document collection for efficient searching.
- LCP Arrays, constructed using the Kasai Algorithm, help identify common textual segments
- The extracted common content is used to determine document similarity and ranking.

Multiple Documents



Suffix Array



LCP + Kasai



Common Text



Similarity & Ranking

Literature Survey

Paper / Work	Method Used	Key Contribution
<i>Manber & Myers (1993)</i>	Suffix Arrays	Introduced an efficient suffix-array based approach for string searching.
<i>Kasai et al. (2001)</i>	LCP + Kasai Algorithm	Proposed linear-time LCP construction from a Suffix Array.
<i>Gusfield (1997)</i>	Suffix Trees	Demonstrated suffix structures for string matching and repeated-substring analysis.
<i>Baker (1995)</i>	Syntactic Pattern Matching	Explored detecting repeated textual patterns for plagiarism-related analysis.
<i>Clough et al. (2002)</i>	Text Matching	Studied automatic plagiarism detection through textual matching techniques.

Problem Statement

- Comparing multiple large text documents to identify duplicate or highly similar content can require repeated textual comparisons. An efficient method is needed to detect common substrings, calculate similarity between documents, and rank the most similar document pairs.

Objectives

- Compare multiple large text documents efficiently.
- Identify common substrings and matching patterns.
- Use Suffix Arrays and LCP Arrays for substring analysis.
- Use the Kasai Algorithm for efficient LCP construction.
- Calculate similarity scores between document pairs.
- Rank documents based on their similarity.
- Evaluate the time, space, and scalability of the approach.

Flow chart

Multiple text documents are provided as input. The text is normalized and prepared for further processing.

The processed documents are represented together while maintaining their individual document identities and boundaries.

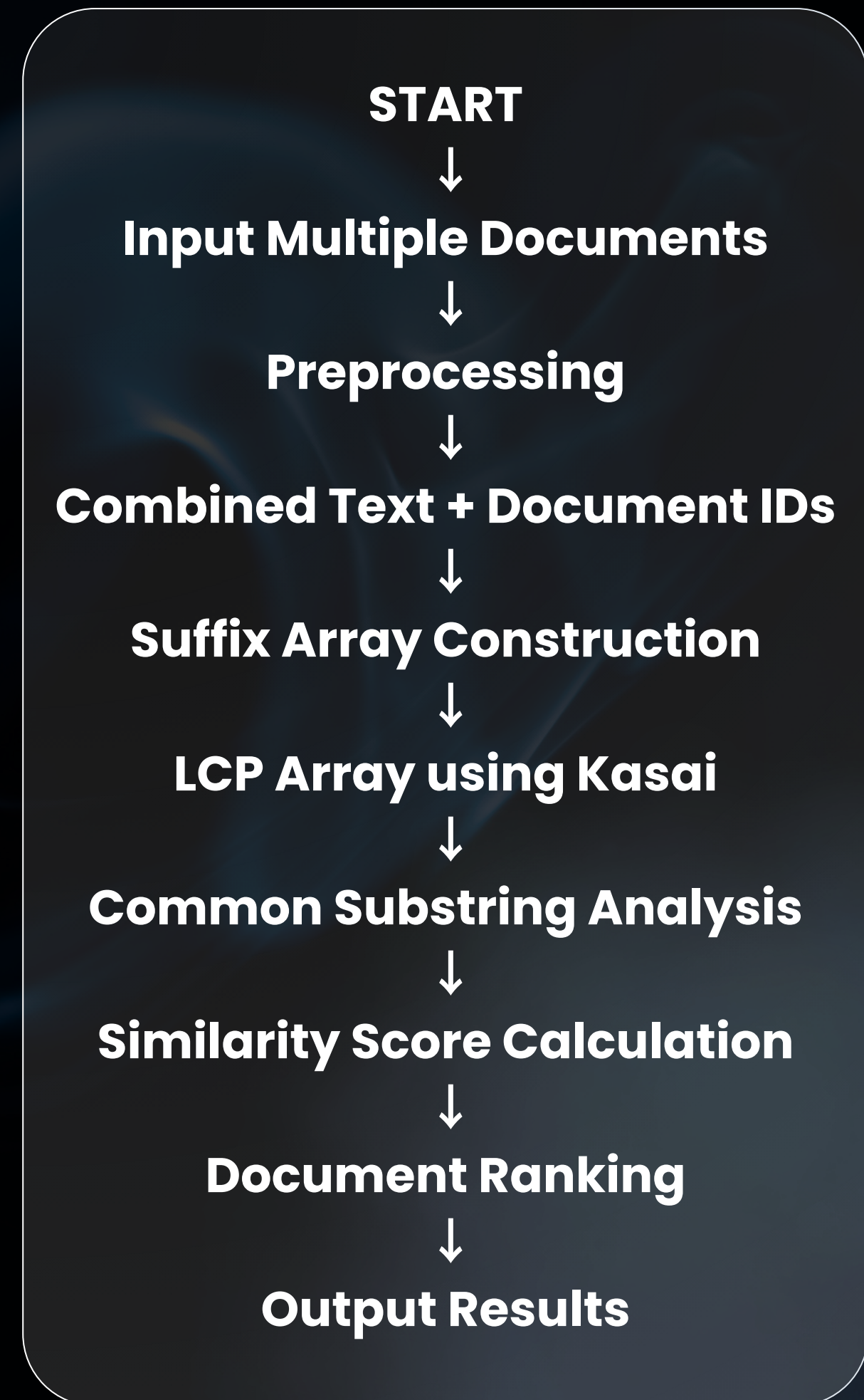
A Suffix Array is constructed to organize the suffixes of the combined text in lexicographical order.

The Kasai Algorithm is used to efficiently construct the LCP (Longest Common Prefix) Array from the Suffix Array. The LCP values indicate the length of common prefixes between neighboring suffixes.

The Suffix Array and LCP information are analyzed to identify common textual segments belonging to different documents.

The shared textual content is used to calculate similarity scores between document pairs. The documents are then ranked from highest to lowest similarity.

The system displays similar document pairs, their similarity scores, and relevant common textual patterns.



Expected Results

1. Common Text Detection

Identify common textual segments shared between different documents.

2. Similarity Scores

Calculate a similarity score for each pair of documents.

3. Document Ranking

Rank document pairs from highest to lowest similarity.

4. Duplicate Detection

Identify documents containing highly similar or duplicate content.

Document Pair	Similarity

Document 1 ↔ Document 2	--%
Document 1 ↔ Document 3	--%
Document 2 ↔ Document 3	--%

Hardware Requirements

- Standard computer/laptop
- Minimum 4 GB RAM
- Sufficient storage for text documents

Software Requirements

- Java JDK
- Visual Studio Code
- Git & GitHub
- Operating System: Windows / macOS / Linux

Core Technologies

Java → Implementation

Suffix Array → Suffix organization

LCP Array → Common-prefix analysis

Kasai Algorithm → Efficient LCP construction

Conclusion

- DOCS_SIMILARITY provides a suffix-based approach for comparing multiple text documents.
- Suffix Arrays organize document suffixes for efficient string analysis.
- Kasai Algorithm efficiently constructs the LCP Array.
- LCP information helps identify common textual segments between documents.
- These shared segments are used to calculate similarity scores and rank document pairs.
- The approach demonstrates the practical application of advanced string algorithms to document similarity.

Future Scope

- Extend the system to detect paraphrased or semantically similar content, not only exact textual matches.
- Support larger document collections with improved scalability.
- Improve the similarity scoring method for more accurate document comparison.
- Add visual reports and similarity graphs for easier result analysis.
- Integrate the system with plagiarism-detection applications.
- Explore parallel processing to speed up similarity analysis for large document collections.



Thank You